Rhino

One queue. Any language. Rust engine.

Self-hosted · Postgres-backed · 3MB binary · Rust, Python, Node · Everything Oban Pro charges $135/mo for — free

1. What is Rhino?

Rhino is an open-source, exactly-once durable job queue for Rust. It ships as a single crate that drops into any Rust project. No broker, no sidecar, no managed service. The only dependency is a Postgres database — which most backend teams already have.

The closest comparison in other ecosystems is Oban for Elixir — a battle-tested, production-grade job processor loved for its simplicity and correctness guarantees. That library does not exist for Rust. Rhino fills that gap.

> **The core guarantee**
> A job enqueued inside a database transaction will execute exactly once — even under worker crashes, network partitions, or simultaneous competing workers. This is not at-least-once. Duplicates are not your problem to solve.

2. The Problem

Every serious backend application eventually needs background job processing: sending emails, charging cards, resizing images, syncing data, firing webhooks. The options today are:

- Hosted SaaS queues (QStash, Upstash, Trigger.dev, Inngest) — you're renting infrastructure indefinitely, your job data lives on someone else's servers, and you're one pricing change or sunset away from an emergency migration
- Roll your own with tokio::spawn — no durability, no retries, no persistence across restarts, jobs vanish on crash
- Port an existing solution — nothing in the Rust ecosystem has the polish, correctness, or ergonomics of Oban

The result: Rust teams either pay forever for hosted queues, accept data loss from in-memory workers, or build something half-baked from scratch. Rhino is the missing library.

3. The Solution — How Rhino Works

The insight

The Postgres feature SELECT FOR UPDATE SKIP LOCKED makes exactly-once delivery possible at the database level. When a worker locks a job row, no other worker can see it. When the worker commits its transaction — marking the job done alongside any business logic changes — both happen atomically or neither does.

This is the Transactional Outbox pattern made first-class and ergonomic. The queue is just a table. The workers are just your app. The guarantee is mathematical.

The jobs table

```
CREATE TABLE rhino_jobs (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    job_type     TEXT NOT NULL,
    payload      JSONB NOT NULL,
    status       TEXT NOT NULL DEFAULT 'pending',
    attempts     INT  NOT NULL DEFAULT 0,
    max_attempts INT  NOT NULL DEFAULT 3,
    run_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    locked_at    TIMESTAMPTZ,
    locked_by    TEXT,
    inserted_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

The API — dead simple by design

Rhino doesn't care what your job does. You write the function. Rhino makes it durable. That's the entire deal.

Step 1 — You already wrote the function. Just add #[rhino].

```
// You had this already
async fn process_order(order_id: u64) {
    // your code — charge card, send email, resize image, anything
}

// This is all floe adds
#[rhino]
async fn process_order(order_id: u64) {
    // exact same code, now durable
}
```

> **One attribute. That's it.**
> Rhino doesn't ask you to learn a job class, a worker class, a serialization format, or a config file. Put #[rhino] on any async function and it becomes durable, retryable, and exactly-once. Nothing else changes.

Step 2 — Start the queue

```
#[tokio::main]
async fn main() {
    let q = rhino!("postgres://localhost/myapp");
    q.start().await;
}
```

Step 3 — Enqueue from anywhere

```
// Minimal — fire and forget
rhino!(process_order(order_id));

// With a handle — batch and send
let q = rhino!();
q.add(process_order(42));
q.add(process_order(43));
q.send().await;

// Scheduled
q.add(send_report()).at(tomorrow());
q.send().await;

// Chained — B runs after A succeeds, C after B
q.add(charge_card(order_id))
 .then(send_receipt(order_id))
 .then(update_inventory(order_id))
 .send().await;
```

The user never sees serialization, job registries, worker loops, or Postgres. They write functions and call send(). Rhino handles everything else.

4. What Happens Under the Hood

Everything below is invisible to the user. They write a job, enqueue it, and it runs. This is what Rhino does on their behalf:

- Job struct is serialized to JSON and inserted into the rhino_jobs table
- A worker loop polls for pending jobs using SELECT FOR UPDATE SKIP LOCKED — this is the exact Postgres primitive that prevents double-execution
- The worker claims a job by setting locked_at and locked_by (worker ID)

- The job's run() method is called inside a transaction
- On success — status set to 'done', transaction commits
- On failure — attempts incremented, exponential backoff applied, retried up to max_attempts
- On max_attempts exceeded — status set to 'dead', moved to dead letter for inspection
- If the worker crashes mid-execution — lock expires, another worker picks it up automatically

5. Why Not a Hosted Queue?

	Hosted Queues (QStash, Upstash)	Rhino
Monthly cost	$20–$500+/month forever	$0 forever
Data sovereignty	Your jobs live on their servers	Never leaves your infra
Delivery	At-least-once (duplicates happen)	Exactly-once, guaranteed
Local dev	Requires internet / mocking	Works offline, always
Compliance	Hard to pass security review	Nothing to approve
Lifespan	Quirrel shut down. IronMQ went paid.	Cannot sunset
Lock-in	Total — migrate or pay	Zero — it's your table

6. Why Not Just Use Oban?

Because Oban is an Elixir library. If you are writing Rust, it is not available to you.

Oban is the inspiration and the gold standard. Rhino is Oban for the Rust ecosystem — which currently has no equivalent. The gap is real, documented, and complained about in Rust forums and Discord servers regularly.

The Rust ecosystem is growing faster than Elixir's. More companies are adopting it for backend systems. The addressable market for a polished, correct, ergonomic Rust job queue is large and completely unoccupied.

6b. One Queue. Any Language. Rust Engine.

Rhino is not just a Rust library. It is a job queue engine that any language can use. The architecture is simple: the Rust worker is the engine, and any language that can talk to Postgres can enqueue jobs into it.

The architecture

```
Your Python app          Your Node app          Your Rust app
      |                       |                       |
pip install rhinoqueue   bun add rhinoqueue    cargo add rhinoqueue
      |                       |                       |
      +-----------------------+-----------------------+
                              |
                     INSERT into rhino_jobs
                     (Postgres — you already have it)
                              |
                     Rhino Rust Worker
                     (single binary, 3MB, 2-5MB RAM)
                     exactly-once  •  retries  •  batches
                     chains  •  workflows  •  rate limiting
```

The worker doesn't know or care which language enqueued a job. It sees a row in a table. It locks it, runs it, commits. The guarantee is universal.

The install experience — v1

Rust

```
# Cargo.toml
[dependencies]
rhinoqueue = "1.0"
```

Python

```
pip install rhinoqueue

# That's it. No compilation. No Rust toolchain. No build step.
from rhinoqueue import Rhino

q = Rhino(os.environ['DATABASE_URL'])
q.add('process_order', {'order_id': 42})
q.send()
```

Node / TypeScript

```
bun add rhinoqueue
# or: npm install rhinoqueue

// No compilation. No Rust toolchain. No build step.
import { Rhino } from 'rhinoqueue'

const q = Rhino(process.env.DATABASE_URL)
```

```
await q.add('process_order', { orderId: 42 }).send()
```

> **How the thin client works**
>
> The Python and Node clients are pure-language packages (~200-300 lines each). They know the rhino_jobs table schema and do exactly one thing: INSERT a row. No Rust runs in your Python or Node process. The Rust worker is a separate binary that drains the queue. You run it once, it handles everything.

Performance across all three languages

The worker performance numbers are identical regardless of which language enqueues. The worker is always Rust. It doesn't know who inserted the row.

```
                     Rust client    Python client    Node client
Enqueue latency      ~0.1ms         ~0.5-2ms         ~0.5-2ms
Worker throughput    Rust speed     Rust speed       Rust speed
Worker RAM           2-5MB          2-5MB            2-5MB
Delivery guarantee   Exactly-once   Exactly-once     Exactly-once

Note: Python/Node enqueue latency is a Postgres INSERT.
Redis round trips on Sidekiq/BullMQ/Celery are 2-3ms.
Rhino's Python and Node clients are as fast or faster to enqueue.
```

What thin client means for v2 — and native bindings for v3

- v2 ships pure Python and pure TypeScript packages — zero compilation, works everywhere psycopg3 or pg runs
- v3 may ship PyO3 and napi-rs native bindings — same install UX, Rust engine running inside the Python/Node process
- The thin client is not a compromise — enqueue performance already beats Redis-based alternatives
- Native bindings only if real production data shows a bottleneck, which is unlikely

6c. Technical Feasibility — Proof

Every technical claim in this project is backed by something that already exists and runs in production. This is not research. This is assembly of proven primitives into a gap that exists.

Core queue — SELECT FOR UPDATE SKIP LOCKED

- Documented, stable Postgres feature since version 9.5 (2016)
- The Postgres documentation uses job queues as the canonical example use case

- Used in production by Oban, Good Job, Que, and dozens of Postgres-backed queues
- Not a hack — the intended usage

Exactly-once delivery

- Mathematical consequence of wrapping job execution in the same Postgres transaction as business logic
- Same pattern as the Transactional Outbox — documented, proven, battle-tested
- Oban does it. Django-pgq does it. That Works (Go) does it.
- Not novel — just not available in Rust yet

The #[rhino] macro

- Rust procedural macros are Turing complete — they can inspect function signatures and emit new code
- #[tokio::main], #[derive(Serialize)], #[instrument] all work this way
- Hard to write well, but completely solved territory in the Rust ecosystem

Thin client packages (Python + Node)

- A pure Python Postgres INSERT wrapper is trivially feasible — psycopg3 + 200 lines
- A pure TypeScript Postgres INSERT wrapper is trivially feasible — pg or postgres.js + 200 lines
- No novel engineering — just schema knowledge and an insert

Native bindings (v3, future)

- PyO3: Pydantic v2 (most downloaded Python lib), Polars, ruff, cryptography.py — all Rust cores via PyO3
- napi-rs: SWC (powers Next.js), Biome, @node-rs/bcrypt — all Rust cores via napi-rs
- maturin and napi-rs CLI handle cross-platform compilation with published GitHub Actions templates
- Not experimental — these are the most downloaded packages in both ecosystems

3MB binary + 2-5MB RAM

- Measurable today — build any Rust project with opt-level=z, strip=true, lto=true
- Rust has no runtime, no GC, no VM — the numbers are a consequence of the language, not a goal
- Tokio async runtime + sqlx connection pool at idle: measured at 2-4MB RSS in existing projects

> **The one honest hard part**
> The #[rhino] macro ergonomics. Getting it to feel truly seamless across all async function signatures, generics, and edge cases takes real work. This won't block v0.1 — the macro can be refined over time. But the DX polish is where most of the hard macro engineering lives.

8. Everything Oban Pro Charges $135/mo For — Free

Oban, the gold standard Elixir queue, splits its features into a free open-source tier and a paid Pro tier at $135/month. Rhino ships all of it for free, forever. Here is what each feature is and how floe provides it:

Batches

Group N jobs together and get a callback when all of them complete. Useful for: fan-out work where you need to know when everything is done — e.g. process 500 images, then send a notification when all 500 finish.

```
let batch = q.batch();
for image_id in image_ids {
    batch.add(process_image(image_id));
}
batch.on_complete(notify_user(user_id)).send().await;
```

Implementation: a batch_id column + completion check query. When the last job in a batch completes, the on_complete job is automatically enqueued. Pure Postgres, no extra infrastructure.

Chains

Job A triggers Job B on success, Job B triggers Job C. A linear sequence of dependent work — if any step fails, the chain stops and retries from that step.

```
q.add(validate_payment(order_id))
 .then(charge_card(order_id))
 .then(send_receipt(order_id))
 .send().await;
```

Implementation: a next_job JSONB field. On successful completion, the worker enqueues the next job in the same transaction. Atomic — either the current job succeeds and the next is enqueued, or neither happens.

Workflows

A DAG (directed acyclic graph) of jobs with arbitrary dependencies. Job C waits for both Job A and Job B to complete before running. More powerful than chains — supports fan-in, fan-out, and complex dependency trees.

```
let wf = q.workflow();
let a = wf.add(fetch_data());
let b = wf.add(fetch_config());
wf.add(process(data, config)).after(a).after(b);
wf.send().await;
```

Implementation: a rhino_workflow_edges join table tracking job dependencies. A job becomes eligible only when all its predecessors have status=done. The hardest feature to build — and the most impressive.

Global and Rate Limiting

Cap how many jobs of a given type run concurrently across all workers globally. Prevent hammering a third-party API, protect a slow downstream service, or enforce business rules like one email per user per hour.

```
#[floe(rate_limit = 10_per_min, global_limit = 5)]
async fn call_stripe_api(amount: u64) { ... }
```

Implementation: Postgres advisory locks + a rate_limit table. Checked atomically at job claim time. No Redis, no external coordination — just Postgres.

Decorators

Middleware hooks that run before and after every job. Add logging, tracing, metrics, auth, or any cross-cutting concern without touching job code.

```
q.use(TracingDecorator::new());
q.use(MetricsDecorator::new());
q.use(RetryOnNetworkError::new());
```

Implementation: a Decorator trait with before_run() and after_run() hooks. Composes as a middleware pipeline. Zero runtime cost — Rust's trait system resolves this at compile time.

Dynamic Plugins

Extend floe's behavior at runtime — custom schedulers, alternative storage backends, observability integrations — without forking the library or redeploying.

```
q.install(DatadogPlugin::new(api_key));
q.install(SlackAlertsPlugin::new(webhook));
```

Implementation: a Plugin trait + runtime registry. Plugins hook into job lifecycle events. Clean Rust trait objects — dynamic dispatch only where needed.

> **The positioning in one line**
>
> Everything Oban Pro charges $135/month for — Batches, Chains, Workflows, Rate Limiting, Decorators, Plugins — Rhino ships free. In a 3MB Rust binary that uses 3MB of RAM.

9. Small, Fast, Efficient by Design

Most job queue libraries are built on top of runtimes, VMs, and garbage collectors. floe is built in Rust — which means none of those things exist at runtime. The efficiency numbers are not a goal, they are a consequence of the language.

Binary size

- Release build with opt-level='z' (optimize for size) and strip=true: 2–5MB total
- A comparable Elixir/Oban node ships the BEAM VM: 40–80MB before your code
- A Node.js queue worker: 50MB+ before your code
- floe: your entire application plus the queue in a single small binary

Memory footprint

- floe worker sitting idle: ~2–4MB RAM
- 10 floe workers: ~40MB RAM
- 10 comparable Elixir/Oban nodes: ~500MB–1.5GB RAM
- No garbage collector means no memory overhead, no GC pauses, no unpredictable spikes

Startup time

- Rust binaries start in milliseconds — no VM warmup, no JIT, no class loading
- Cold starts are irrelevant — deploy to serverless, containers, or spot instances freely
- CI test suite starts instantly — no waiting for a runtime to initialize

Zero-cost abstractions

The #[rhino] macro, the plugin system, the decorator pipeline, the Job trait — in Rust these all compile away. You pay nothing at runtime for the abstractions. The generated machine code is as fast as if you had written everything by hand with no library at all.

Cargo.toml profile for production

```
[profile.release]
opt-level = "z"      # optimize for binary size
strip = true          # strip debug symbols
lto = true            # link-time optimization
codegen-units = 1     # maximize optimization
```

10. Technical Stack

Purpose	Crate	Why
Async runtime	tokio	Industry standard async runtime for Rust
Postgres driver	sqlx	Async, compile-time query verification
Serialization	serde + serde_json	Universal — users bring their own types
Job IDs	uuid	Collision-resistant unique identifiers
Timestamps	chrono	Timezone-aware scheduling and backoff
Error handling	thiserror	Clean, typed errors for a library crate

11. Roadmap

v0.1 — Core (working prototype)

- #[rhino] macro on any async function — the entire public API
- rhino!() macro — zero-config queue handle from env
- q.add() / q.send() interface
- Core jobs table + auto-migration runner
- Worker loop — SELECT FOR UPDATE SKIP LOCKED
- Basic retry with exponential backoff
- Dead letter queue

v0.2 — Power features (free, everything Oban Pro charges for)

- Chains — .then() API, next job enqueued in same transaction
- Batches — q.batch() + on_complete() callback
- Scheduled jobs — .at() and .in() on any job
- Priority queues — priority field, ORDER BY in worker query
- Unique jobs — deduplication by function + args hash
- Decorators — q.use() middleware pipeline
- Rate limiting — #[floe(rate_limit)] attribute, Postgres advisory locks
- Global concurrency limits — cap parallel execution globally

v0.3 — Workflows + production hardening

- Workflows — DAG dependencies, rhino_workflow_edges table
- Dynamic plugins — q.install() trait object registry
- Graceful shutdown — drain in-flight jobs before exit
- Configurable worker concurrency — N workers per queue
- Comprehensive test suite — exactly-once proof under concurrent workers

v1.0 — Ecosystem launch

- Published to crates.io
- Full documentation on docs.rs with examples for every feature
- Optional web dashboard — q.dashboard() mounts at /rhino, zero config
- Benchmark suite — binary size, RAM, throughput vs Oban, BullMQ, Sidekiq
- Release build profile — 2-5MB binary, documented and verified

v2.0 — Multi-language (thin clients)

- rhinoqueue on PyPI — pure Python, pip install rhinoqueue, zero compilation, zero Rust toolchain
- rhinoqueue on npm — pure TypeScript, bun add rhinoqueue or npm install rhinoqueue, zero compilation
- Both clients are ~200-300 lines — they know the rhino_jobs schema and do a Postgres INSERT
- Python and Node apps enqueue jobs — the Rust worker processes all of them from the same table
- Enqueue performance: ~0.5-2ms per job from Python or Node — same as or faster than Redis
- No change to the Rust worker — it already handles jobs from any producer

v3.0 — Native bindings (future, maybe)

- PyO3 native extension — Rust engine compiled into a .so, imported by Python natively
- napi-rs native addon — Rust engine compiled into a .node file, imported by Node natively
- Same pip install / bun add UX for users — pure performance upgrade under the hood
- Proven path: Pydantic v2, Polars, ruff all use PyO3. SWC and Biome use napi-rs.
- Only ships if thin client shows a real bottleneck — unlikely since the worker is always Rust

12. What Rhino Is Not

- Not a message broker — no pub/sub, no fan-out, no competing consumers across services
- Not a streaming platform — not Kafka, not a replacement for event streaming
- Not a cron scheduler — though scheduled jobs are supported, Rhino is not a cron daemon
- Not a SaaS — there is no Rhino cloud, no Rhino dashboard to log into, no account required
- Not magic — Rhino relies entirely on Postgres. If your Postgres is down, your queue is down. This is a feature: one fewer thing to monitor.

13. Who Is This For?

- Rust backend teams who already run Postgres and need durable background jobs
- Teams who have been burned by hosted queues sunsetting or changing pricing
- Fintech, healthcare, or any regulated industry where job data cannot leave the application's infrastructure
- Engineers who want to understand and own every line of their job processing stack
- Anyone building a Rust service who wants the Oban experience without switching languages

> **Resume signal**
> Building floe demonstrates: async Rust, database internals (WAL, MVCC, row-level locking), distributed systems correctness, open source library design, and the ability to ship something real people use. This is the kind of project that gets a response from Databricks, Cloudflare, or a FAANG infra team.

14. Performance Benchmarks — Industry Standards vs floe Targets

All competitor numbers below are sourced from official benchmarks and documented production data. floe targets are conservative estimates based on Rust's measured performance characteristics versus these runtimes. All throughput numbers use no-op or near-zero-work jobs to measure raw queue overhead.

> **How to read this table**

> Throughput = raw jobs/sec with no-op work, single node. Idle RAM = worker process memory at rest, no jobs. Startup = time from binary execution to ready. Binary = total deployable artifact size. Delivery = the actual delivery guarantee, not the marketing claim.

Throughput & Resource Comparison

Metric	Sidekiq (Ruby/Redis)	BullMQ (Node/Redis)	Oban (Elixir/PG)	Celery (Python)	floe (Rust/PG) [TARGET]
Peak throughput (no-op jobs, 1 node)	~30K jobs/sec (96 cores, Dragonfly)	~27K jobs/sec (concurrency=100)	~17K jobs/sec (concurrency=230, M1 Pro)	~1K–3K jobs/sec (Celery + RabbitMQ)	**~20K–40K jobs/sec (projected, 10 workers, PG on same machine)**
Typical throughput (10 workers, real hw)	1K–5K jobs/sec	8K jobs/sec (concurrency=100)	4.4K jobs/sec (concurrency=100)	500–1.5K jobs/sec	**5K–15K jobs/sec (projected)**
Idle RAM (1 worker process)	~180–500MB (Rails loaded)	~80–150MB (Node.js + V8)	~50–150MB (BEAM VM)	~60–120MB (Python runtime)	**~2–5MB (no runtime)**
RAM — 10 workers	1.8GB–5GB	800MB–1.5GB	500MB–1.5GB	600MB–1.2GB	**~20–50MB (projected)**
Binary / deploy size	Rails app 50MB–200MB+	Node app 50MB–150MB+	Elixir release 30–80MB	Python env 100MB–300MB+	**~2–5MB (single binary)**
Startup time	2–10 seconds (Rails boot)	500ms–2s (Node.js)	1–3 seconds (BEAM boot)	1–5 seconds (Python import)	**<50ms (native binary)**
Delivery guarantee	At-least-once (duplicates possible)	At-least-once (Redis AOF risk)	At-least-once (exactly-once optional with Pro)	At-most-once (default) or at-least-once	**Exactly-once (SELECT FOR UPDATE SKIP LOCKED)**
Extra infra required	Redis (required)	Redis (required)	PostgreSQL	RabbitMQ or Redis	**PostgreSQL only (you already have it)**
Language	Ruby	Node.js	Elixir	Python	**Rust**
Monthly cost (hosted/licensed)	Free OSS (Redis infra cost)	Free OSS (Redis infra cost)	Free OSS +$135/mo Pro	Free OSS (broker infra cost)	**Free. Forever. No license. No broker.**

Notes on the Numbers

- Sidekiq peak throughput figure is from the official Dragonfly benchmark (96-core EC2, 800K jobs in 26s). Real production throughput on typical hardware is 1K–5K jobs/sec.
- BullMQ throughput from the official BullMQ vs Oban benchmark (27.2K peak, 8.3K at concurrency=100 with 10ms work).
- Oban throughput from the official Oban Pro benchmark post: 17,699 jobs/sec peak on Apple M1 Pro, concurrency=230. Typical single-worker production: 4.4K jobs/sec at concurrency=100.
- Sidekiq idle RAM: ~180MB RSS observed in production GitHub issues; can reach 500MB+ under load with Rails stack.
- floe throughput target is conservative — Rust async I/O with Tokio and Postgres SELECT FOR UPDATE SKIP LOCKED is the same stack Oban uses (PG), but without the BEAM VM overhead. We target 2–4x Oban's throughput at equivalent concurrency.
- floe RAM target: a Tokio async runtime with a PG connection pool and no garbage collector realistically idles at 2–5MB. This is measurable and will be published with the v1.0 benchmark suite.
- All floe targets are projections until the benchmark suite ships with v1.0. We will update this table with real numbers.

> **The number that matters most**
>
> RAM at 10 workers: ~20–50MB for floe vs 500MB–5GB for every competitor. On a $10/mo VPS, you can run floe alongside your entire application. The same setup with Sidekiq or BullMQ requires a dedicated worker server.

Stress Test Plan — How We Verify These Numbers

floe ships with a benchmark suite in benches/. Every metric in the table above has a corresponding test. Nothing is published without being measured.

Throughput test (criterion)

```
// benches/throughput.rs
// Enqueue 10,000 no-op jobs, drain with N workers
// Measure: jobs/sec at N = 1, 5, 10, 25, 50 workers
// Report: p50, p95, p99 end-to-end latency
```

Exactly-once contention test (correctness)

```
// tests/exactly_once.rs
// Enqueue 10,000 jobs
// Spawn 20 workers simultaneously
// Each job: atomically increment a Postgres counter
// After drain: assert!(counter == 10_000)
// assert!(dead_letter_count == 0)
// FAIL = any job ran twice
```

Memory test

```
// scripts/measure_memory.sh
// Start worker process
// Record RSS at: idle, 100 jobs/sec, 1000 jobs/sec
// Tool: /proc/self/status on Linux, ps on macOS
```

Binary size test

```
# scripts/measure_binary.sh
cargo build --release
ls -lh target/release/floe
# Target: < 5MB
```

Crash recovery test

```
// tests/crash_recovery.rs
// Start job with 5s sleep
// Kill worker process at 2s (mid-execution)
// Start new worker
// Assert: job re-runs and completes
// Assert: lock_timeout triggers within configured window
```

15. Agent Reference — Implementation Notes

This section is for AI agents or developers picking up this codebase mid-stream.

Repository structure

```
rhino/
├── Cargo.toml
├── src/
│   ├── lib.rs          # Public API surface — Floe struct, Job trait
│   ├── worker.rs       # Worker loop, polling, locking
│   ├── queue.rs        # Enqueue logic, serialization
│   ├── job.rs          # Job trait definition
│   ├── error.rs        # thiserror error types
│   ├── config.rs       # RhinoConfig — concurrency, timeouts, backoff
│   └── migrations/
│       └── 001_create_rhino_jobs.sql
├── tests/
│   └── integration.rs  # Requires running Postgres
└── examples/
    └── basic.rs        # Minimal working example
```

The locking query — do not change without understanding

This is the heart of the exactly-once guarantee. Do not simplify it.

```
SELECT id, job_type, payload, attempts
FROM rhino_jobs
WHERE status = 'pending'
  AND run_at <= NOW()
```

```
ORDER BY priority DESC, run_at ASC
LIMIT 1
FOR UPDATE SKIP LOCKED;
```

SKIP LOCKED means: if another worker already holds a lock on this row, skip it and move to the next. This prevents double-execution without any application-level coordination.

Retry backoff formula

```
// Exponential backoff with jitter
// attempt 1 -> ~30s, attempt 2 -> ~4m, attempt 3 -> ~30m
fn next_run_at(attempts: i32) -> DateTime<Utc> {
    let base: i64 = 15 * 2_i64.pow(attempts as u32);
    let jitter: i64 = rand::thread_rng().gen_range(0..base / 2);
    Utc::now() + Duration::seconds(base + jitter)
}
```

Job registry — how dynamic dispatch works

The registry maps job_type strings (stored in Postgres) back to concrete Rust types at runtime. Each registered type implements Job and knows how to deserialize itself from the JSONB payload.

```
// Internal registry — users never touch this directly
type JobRunner = Box<dyn Fn(&str) -> BoxFuture<Result<()>> + Send + Sync>;

pub struct Registry {
    handlers: HashMap<String, JobRunner>,
}

// #[rhino::job] macro generates the registration glue
// job_type = "process_order" (type name as string)
```

Key invariants — never violate these

- A job row transitions: pending → (locked) → done | failed | dead. Never backwards.
- locked_by must be set atomically with locked_at in the same UPDATE as the SELECT FOR UPDATE
- The worker must always release or commit its transaction — never hold an open transaction across non-DB work
- max_attempts=0 means unlimited retries. Default is 3.
- Dead letter jobs are never deleted automatically — they are for human inspection

Testing approach

- Unit tests: job serialization, backoff formula, config parsing — no DB needed
- Integration tests: require a real Postgres instance (use Docker in CI)

- Key integration tests: exactly-once under concurrent workers, retry on failure, dead letter after max_attempts, crash recovery (kill worker mid-job, verify pickup)